Part 1

1. **Row Key Sharding:**
   The proposed row key for this dataset is City#Make#VIN(1-10). City works well as the leading component because Washington has hundreds of distinct cities, so rows naturally spread out across different tablet servers instead of piling onto one, avoiding hotspotting. Since Bigtable sorts everything by row key, if we used a timestamp or DOL Vehicle ID instead, all new writes would land at the end of the key space and hit the same tablet. With City in front, new EV registrations are happening in Seattle, Tacoma, Spokane, and dozens of other cities at the same time, so the writes get distributed across the cluster. The VIN(1-10) at the end makes sure every row key is unique, even if two cars have the same city and make. This key also makes querying by city more efficient as scanning for all Seattle vehicles is just a prefix scan on "Seattle#" that only touches the relevant tablets instead of scanning the whole table.

2. **Column Families:**
   I would split the dataset into four column families, identity (VIN, DOL Vehicle ID, County, City, State, Postal Code) for registration lookups; vehicle (Make, Model, Model Year, Electric Vehicle Type, Clean Alternative Fuel Vehicle Eligibility) for analytical queries; performance (Electric Range) as a standalone numeric metric for filtering and aggregation; and geographical (Legislative District, Vehicle Location, Electric Utility, 2020 Census Tract) for policy and geographic analysis.

   This aids in distributed read performance because Bigtable stores each column family in separate SSTables on disk. A query that only needs vehicle specs reads only the vehicle data, avoiding irrelevant I/O across 200,000+ records.

3. **Horizontal Scaling:**
   A hashed shard key on VIN allows for almost perfect, uniform distribution across shards since every VIN is unique. This gets rid of write hotspots and maximizes cluster utilization. Point lookups by VIN route to exactly one shard. However, hashing gets rid of ordering, so any query that isn't an exact VIN match must scatter-gather across every shard, which increases latency.

   A range-based shard key on Model Year allows for efficient range scans over time by targeting only the relevant shards. But it suffers from severe data skew for this dataset, EV adoption grew exponentially, so recent-year shards hold more records and take up disproportionate traffic while early-year shards sit idle. For balanced performance, a compound key like { Model_Year: 1, VIN_hashed: 1 } can combine range-scan benefits with even distribution.

Part 2

4. **Performance Analysis:**

```python
import time, requests

URL = "http://localhost:5000"
PAYLOAD = {"Make": "TESLA", "Model": "Test", "Model Year": "2024", "City": "Seattle", "State": "WA"}

def bench(endpoint):
    times = []
    for i in range(50):
        start = time.time()
        requests.post(f"{URL}{endpoint}", json=PAYLOAD)
        times.append(time.time() - start)
    avg = sum(times) / len(times) * 1000
    print(f"{endpoint}: avg {avg:.2f} ms over 50 requests")
    return avg

fast = bench("/insert-fast")
safe = bench("/insert-safe")
print(f"\n/insert-safe is {safe/fast}x slower than /insert-fast")
```

insert-fast: avg 31.81 ms over 50 requests
insert-safe: avg 33.95 ms over 50 requests
/insert-safe is 1.0672915142648134x slower than /insert-fast

A good example would be something like a ride-sharing app like Uber which tracks driver locations. Drivers send their GPS coordinates every few seconds, so you're getting a massive flood of writes constantly. If the primary node crashes and you lose some location pings, it doesn’t matter as much because the driver's phone sends a new one two seconds later. Using w=1 here makes sense because we prioritize availability and low latency over strong consistency. If we used w="majority" instead, every GPS ping would have to wait for two out of three nodes to confirm it, slowing things down. When handling thousands of drivers, this extra latency quickly accumulates and could cause the system to fall behind